

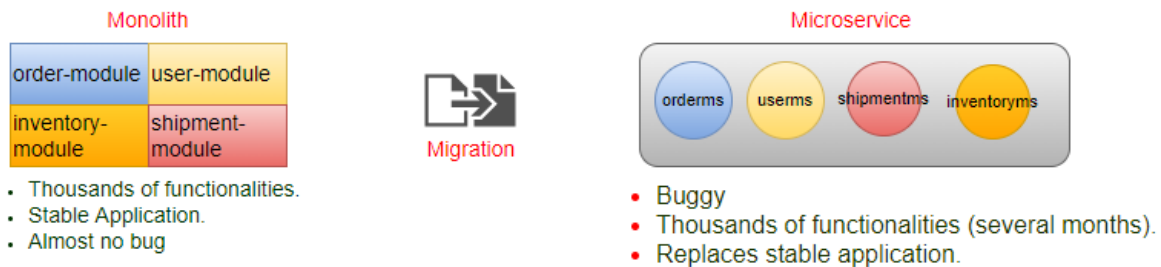
Untangling the ball of Mud (Migrating from Monolith)

BigBang Approach

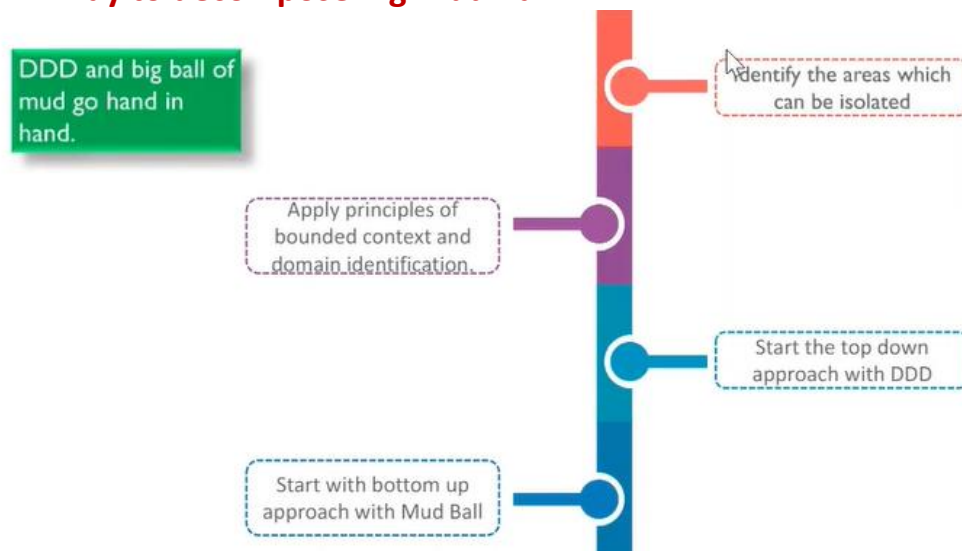
Migrate complete monolith in First Day of Release, is not recommended

Migration Strategy

Big Bang Approach (Not Recommended)



DDD way to decompose Big Mud Ball



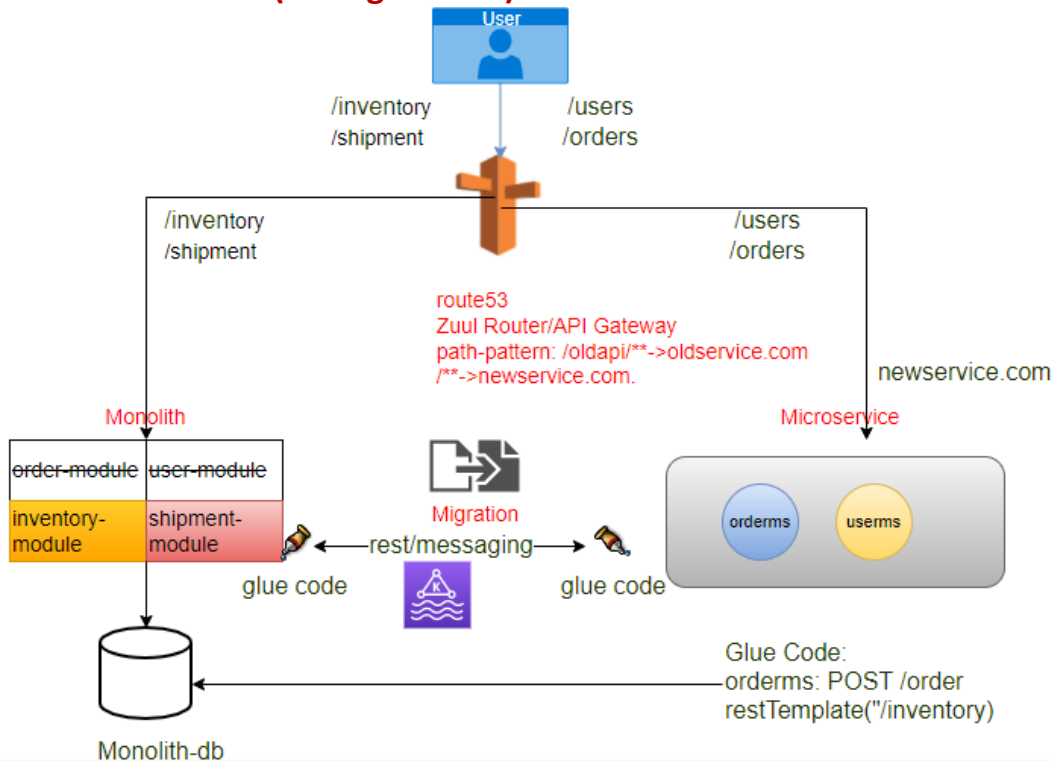
Strangler Vine Approach (Hybrid): Says start from a one simple module, and convert it into ms after DDD (do the refactoring at monolith side before the migration).



A strangler vine grows around a tree in order to reach the sunlight above the forest canopy. Sometimes, the tree dies, leaving a tree-shaped vine.

Application modernization follows the same pattern. We will build a new application consisting of Microservices around the legacy application, which will eventually die.

Hybrid Architecture (Strangler Vine):



Is hybrid architecture

Once we move **usersms** from monolith to ms, remove the code of user ms from monolith side (size of monolith code reduces).

UC: Whether we hit to monolith or to ms?

Through API Gateway (URL hiding)

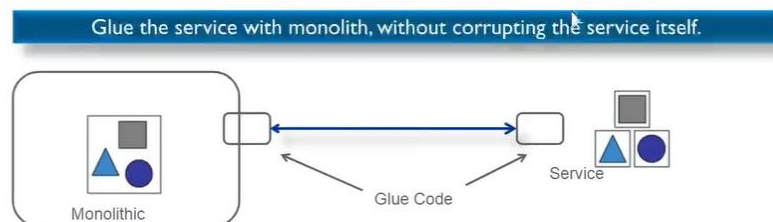
Note: migrate small-small modules(functionality) step-by-step (Iterative Incremental). Our aim to reduce the size of monolith, if any major change or new enhancement come will be done at microservice side.

UC: How ms access the database?

Glue Code (External Library): It may be Restful logic or through Messaging Refactoring at monolithic side (convert method calls to remote call).

How Glue Code looks like?

Glue Code



WHY?

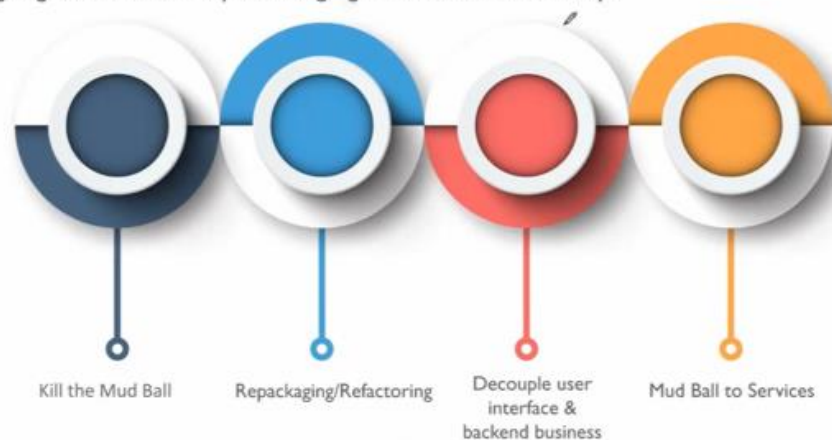
It is needed mostly as services will have some dependencies on the monolith, which must be preserved until the transition to Microservices is complete.

Glue code helps in building services independently and still use the monolith until necessary.

When the monolith dies, the service could be used as it is without much impact.

Untangle Monolith-Strategies

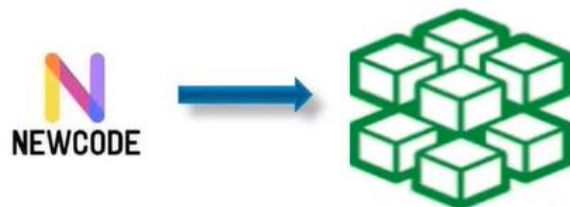
The strategies given below can help in untangling a monolith in various ways.



How to stop the Monolith from Growing

01 When a new functionality is being implemented excessive code must not be added to the monolith.

02 Instead, the big idea with this strategy is to put that new code in a standalone microservice.



Adding a Microservice to mud ball

STEP
01

Build a standalone microservice

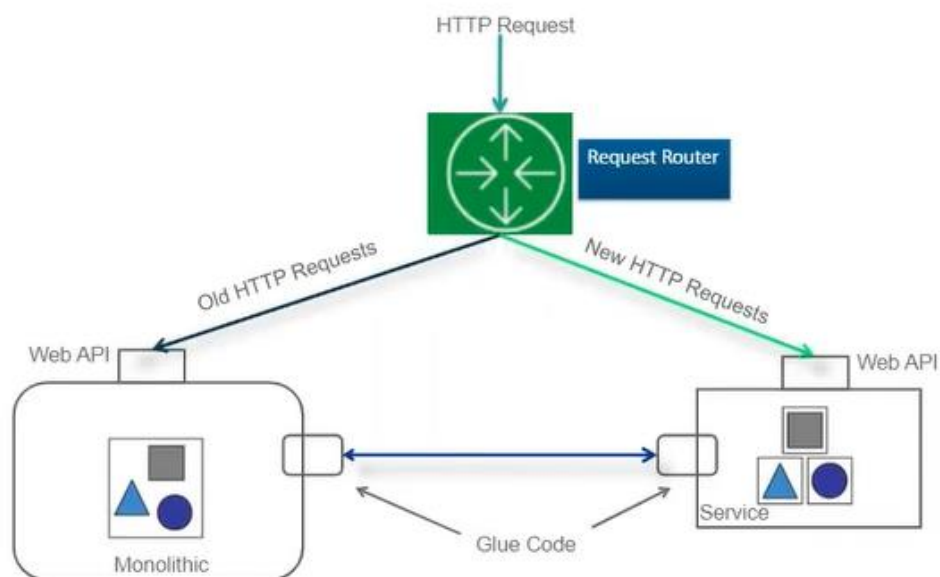
STEP
02

Connect the monolith to the microservice

STEP
03

Facilitate data flow between the two

New System Architecture



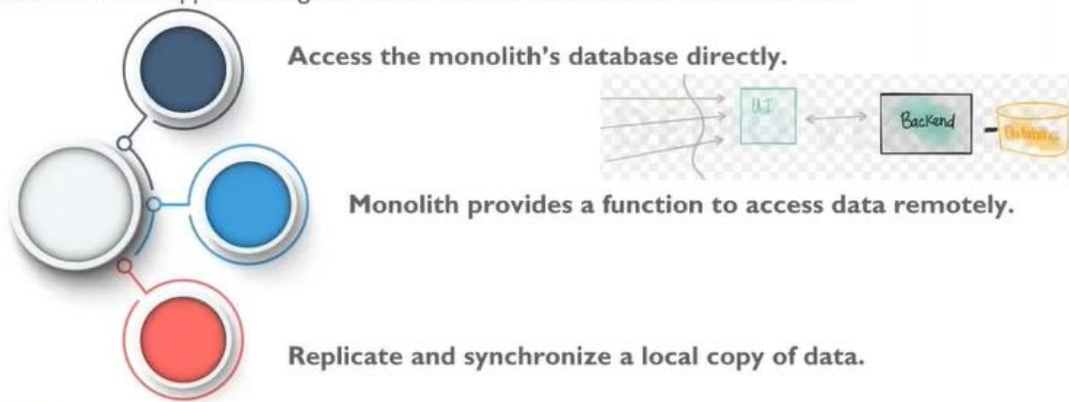
Data Access

Hybrid DB Architecture (Who is owner of DB): Only owner has access of write operations

- Access the monolith's database directly.
- Monolith provides a function to access data remotely.
- Replicate and synchronize a local copy of data.

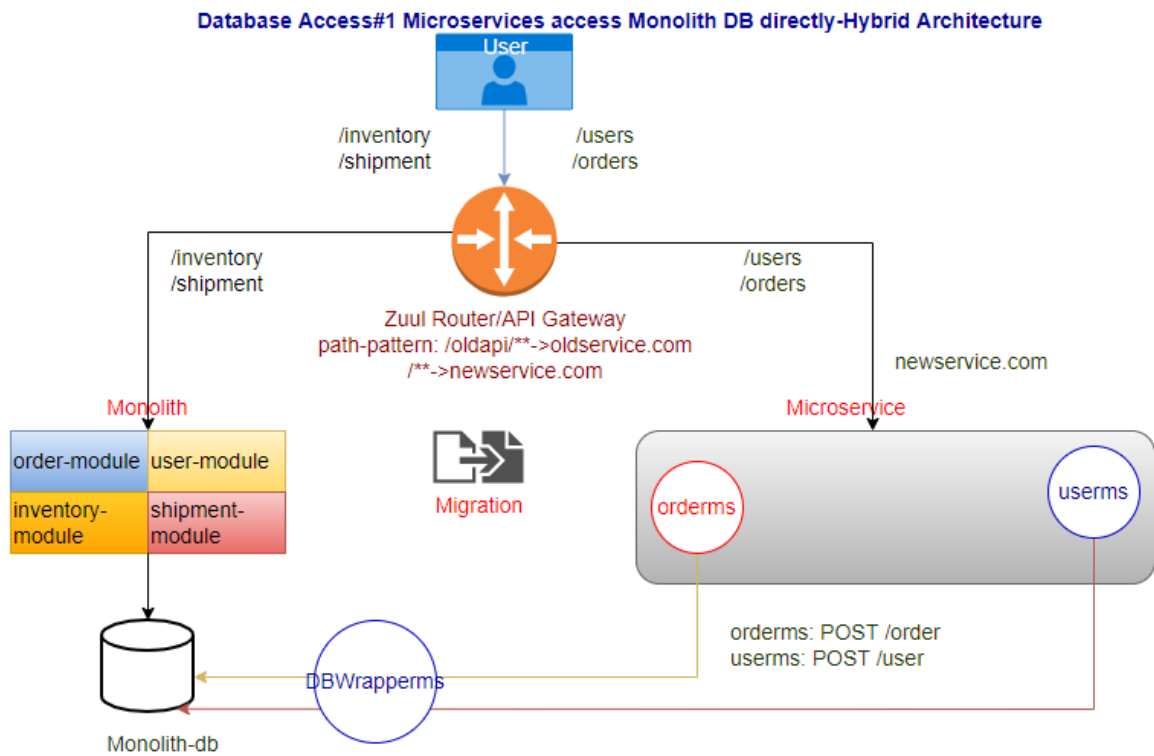
▪ For accessing the data from monolith, though the microservices philosophy believes in data encapsulation with the service itself, it might not be possible in all scenarios.

▪ In such cases one of the approaches given below could to be used as an interim solution



UC1: Access the monolith's database directly.

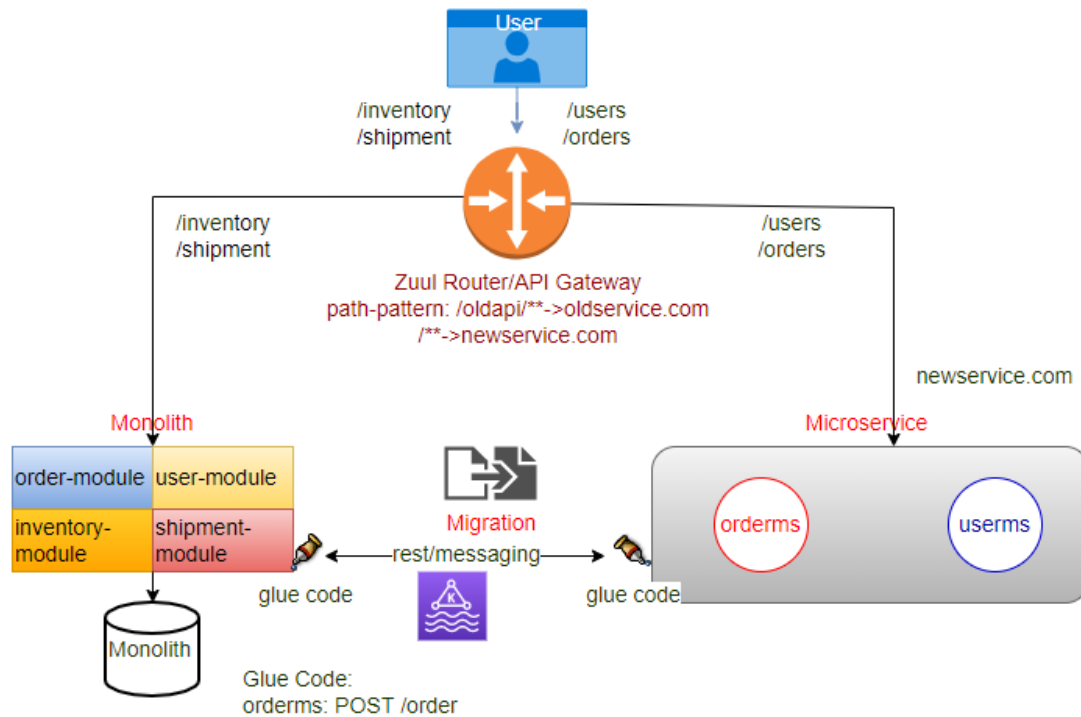
Using DBWrappers, single point of access from ms side.



Limitation: Data Modeling, if there are data coupling between order and user db.

UC2: Access the monolith's database through monolith.

Database Access#2 Microservices access Monolith DB through Monolith-Hybrid Architecture

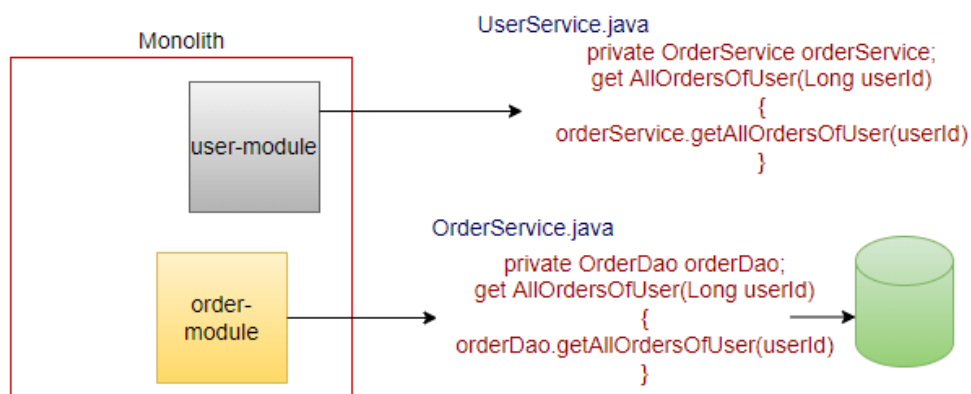


Limitation: ms calls REST API of monolith (changes in monolith side), later on once app will be migrated fully require changes in ms side (remove the call).

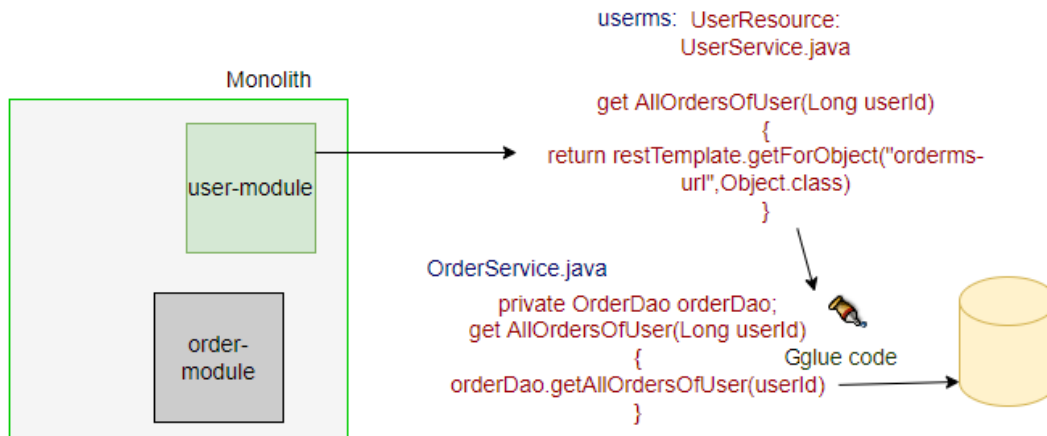
Note: In both above case there is a single source of truth (monolith).

UC: user-module needs all order-modules details

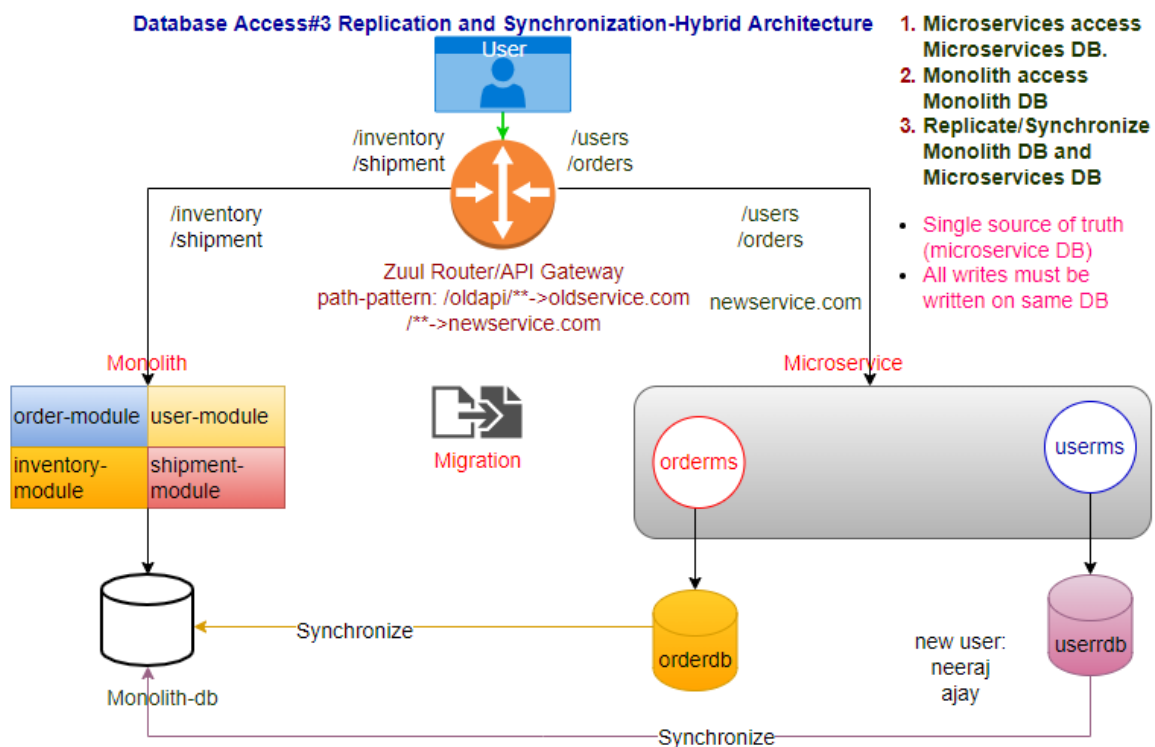
In monolith



UC: we have taken user service module taken out and moved to usersms
 First, we have to refactor the code (userservice.java) in monolith will not exist.



UC3: Replicate and Synchronize.



step1: New user Ajay comes and registered himself in usersms-db.

step2: Ajay wants to place order in monolith application, but user was created in usersms-db not in monolith db.

How to ensure data must be in sync?

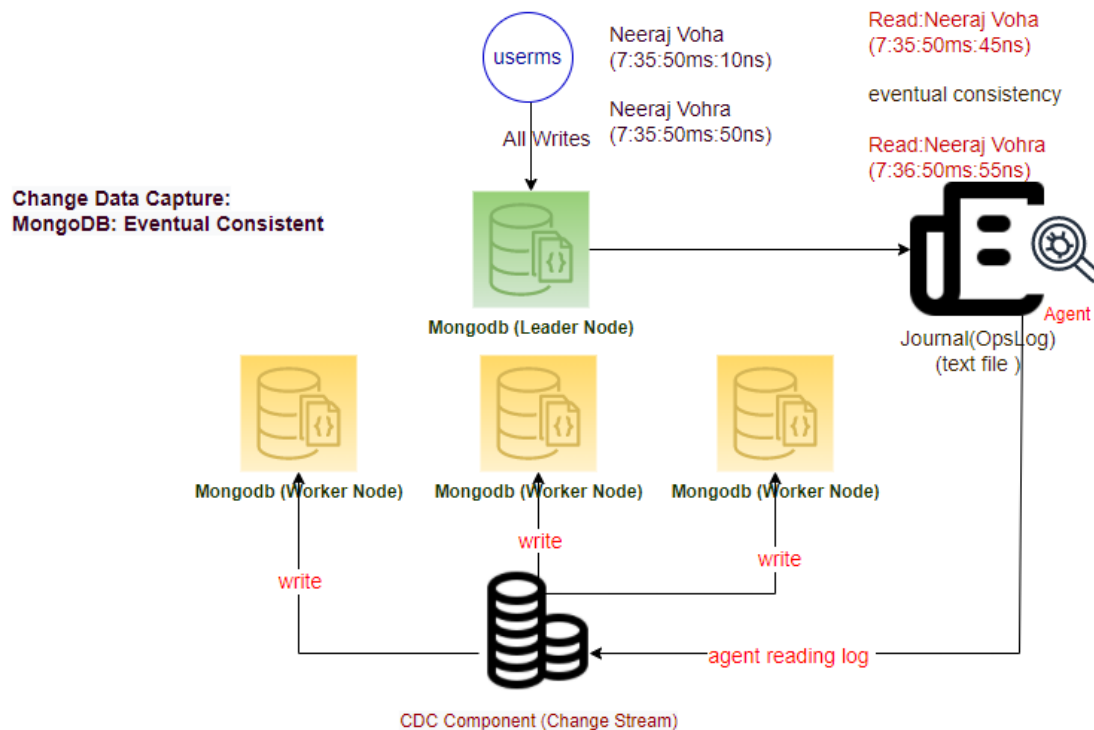
Monolith and ms must be replicate and synchronized.

Always there will be single source of truth, you must be synchronized either from users-db to monolith db, or synchronize from monolith db to users-db but never from both sides. Never two-way synchronization (RACE condition).

We must have to keep ordersms-db as a single source of truth for ordersms

We have to replicate the ordersms-db to order db of monolith.

CDC (Change Data Capture): Eventual Consistency



CAP Theorem:

- Consistency
- Availability
- Partition Tolerance

Distributed Database MongoDB:

- Distributed DB maintain a Journal In mongo db it's called OpsLog
- Whenever leader node receives any write operation (write/update/delete) it writes the same into Journal (OpsLog).
- Using this Journal all updates are recorder line-by-line, every update must be added here.
- These updates must be traveled to worker nodes, in case of distributed mongodb.

- There is another component CDC component (agents running on CDC component reading the log).
- Whatever it changes receives from OpsLog writes to the worker nodes.
- Mongo db not supports consistency property of CAP Theorem, It **Eventually consistent**.
- In case of RDBMS provides the **Immediate Consistency**, while it's not supporting Partition Tolerance. RDBMS not supports CDC because they don't support distributed db architecture.
- Read Only through the worker-node.
- Data resides in cluster of nodes.
- Data writes on single leader node.

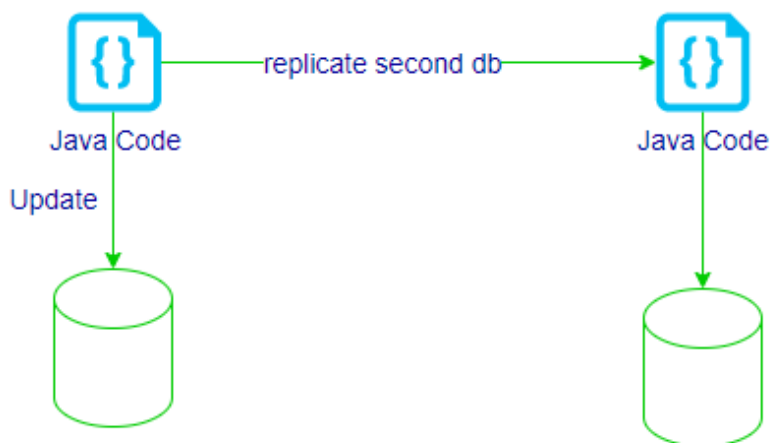
How to replicate?

UC1: Call service to replicate

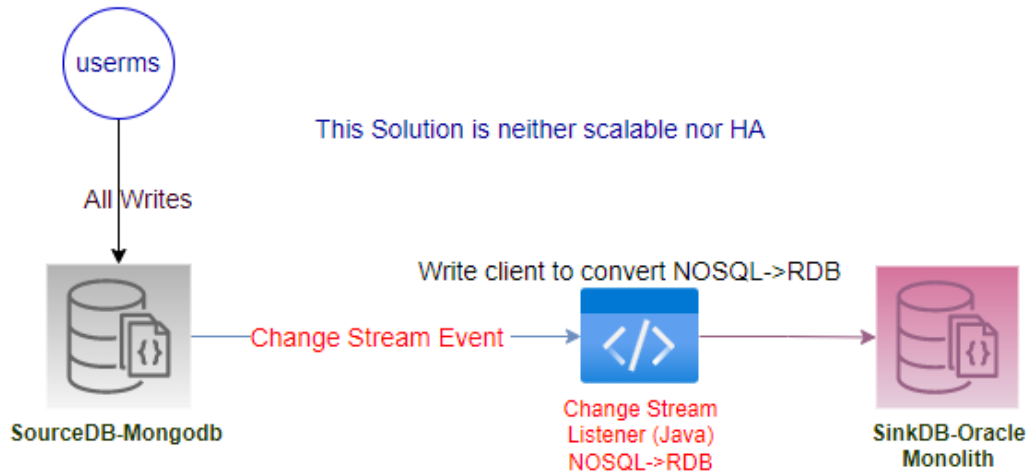
step1: update the db

step2: call service to replicate other DB

step3: commit the change

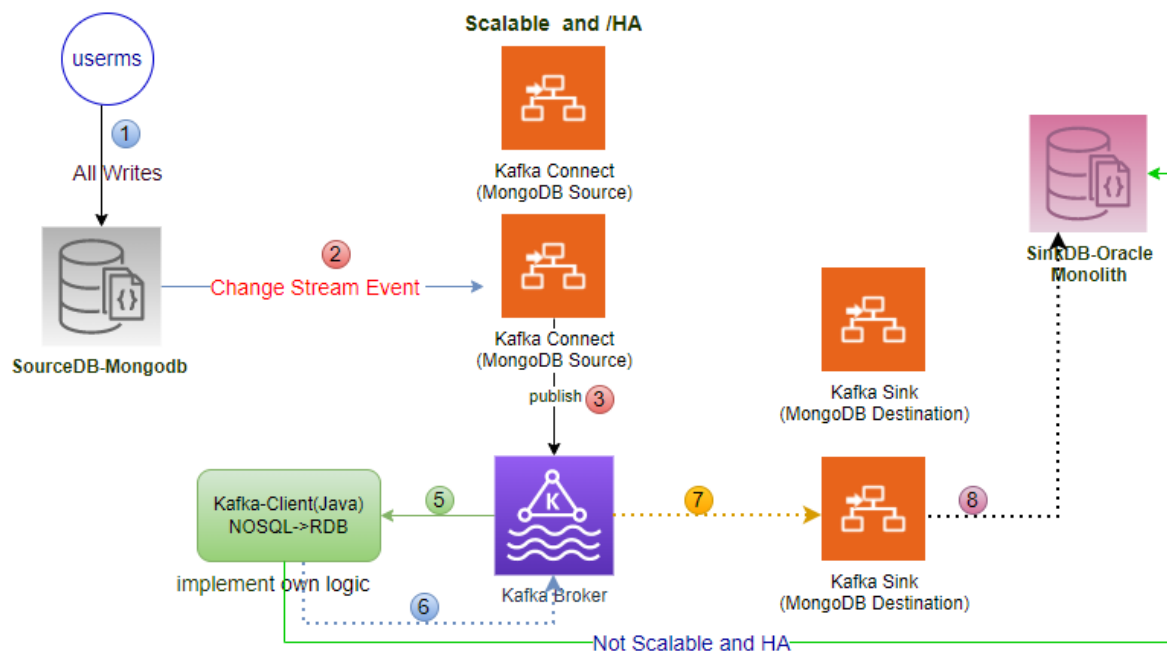


UC2: Write your client (listen to the CDC components), convert schema and write to other DB.



UC3: Using DB provided CDC feature

We plug-in our component with-in CDC component, and through that component we replicate our database.



Kafka Connect is out of box solution, provided by Kafka, **Kafka Connect** Job is consuming the messages from Change Stream Event and publish to Kafka, don't need to write them just configure the same.

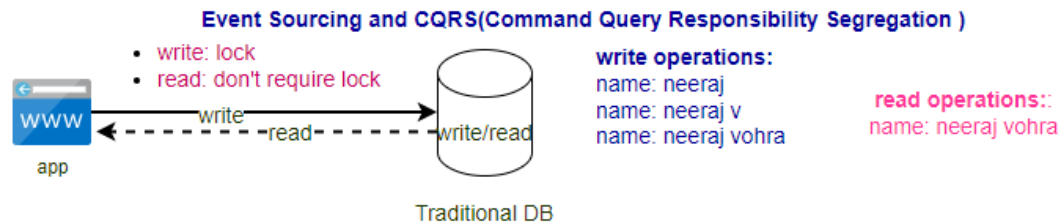
In case of Oracle, use another Out of Box Solution -Debezium Connector for Oracle DB, Use Debezium Connect for Oracle.

kafka is meant for low-latency tool: Gives 7 days persistent by default

Event Sourcing and CQRS

UC: Extremely high writes

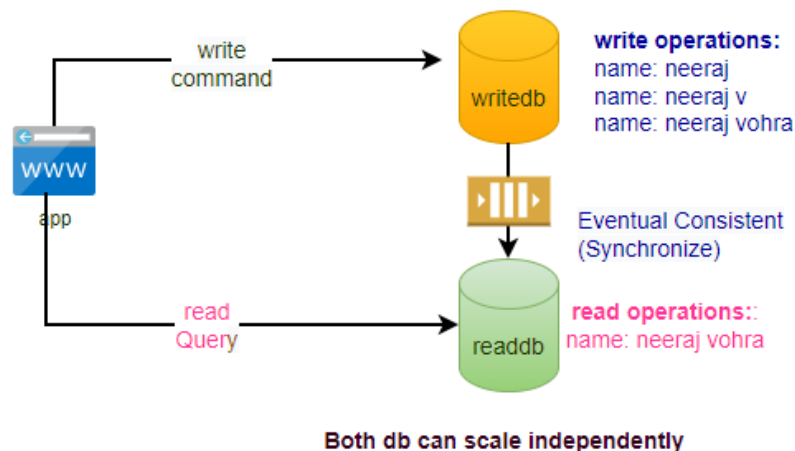
Event sourcing, we track every event the journey of your data.



CQRS: Command Query Responsibility Segregation

UC: Extremely High Writes

CQRS: Command Query Responsibility Segregation

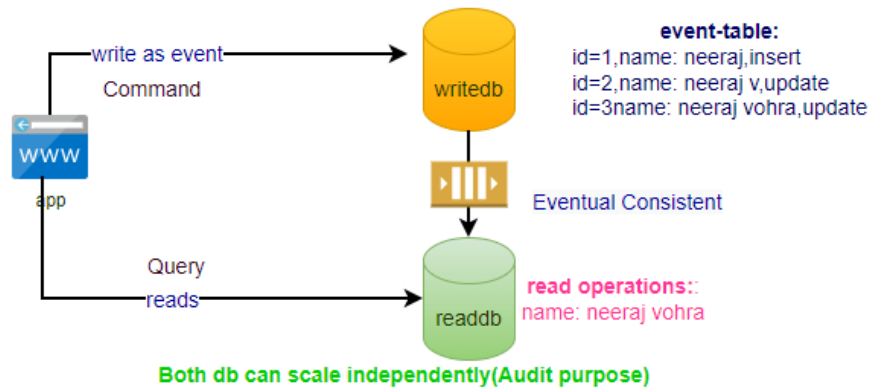


Event Sourcing:

- In tradition database approach **destination** was more important than **journey**.
- In Event Sourcing **Journey** is more important than **destination**.
- It is very important for **auditing purpose**.
- It takes any update as an event.
- Event sourcing is also possible in Oracle DB.
- limitation is that can't query the data.
- Data is Eventual consistent.

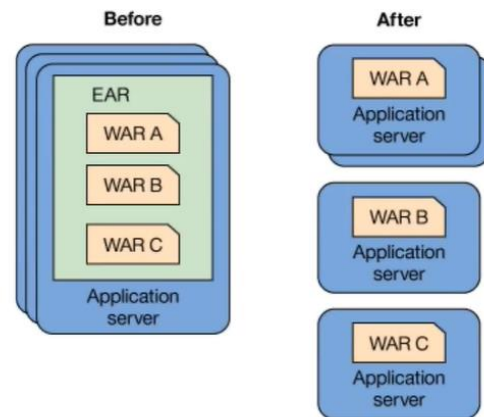
Limitation of event table can't query the data used with CQRS.

Event Sourcing + CQRS:



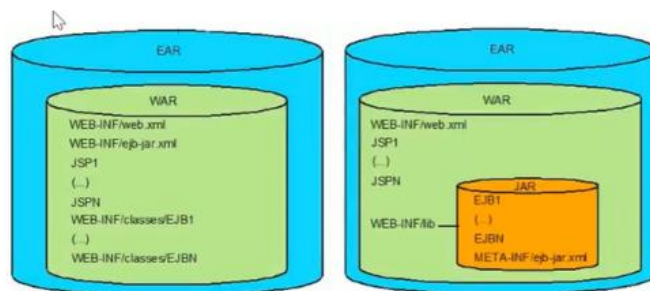
Repackaging (specific for enterprise applications)

- Both refactoring and repackaging are related to managing monolith deployment while it exists.
- This usually is a coarse grained approach, like splitting up an EAR into multiple WARs.
- Once split, these can then be built, deployed and managed independently. This applies more to the J2EE solutions.



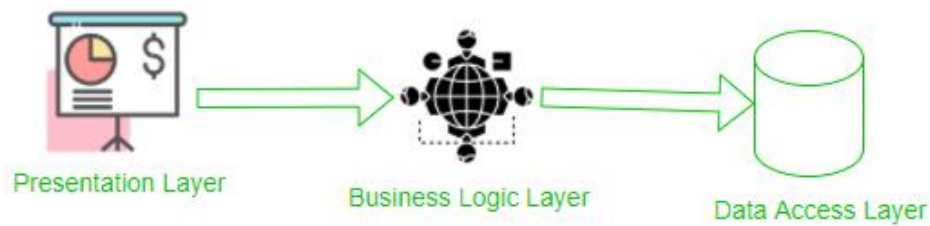
Refactoring Gradually

- Fine grained approach
 - Splitting up a war into multiple jars – and
 - Involves moving code into logical packages and modules
 - This is generally associated with creating well defined API interfaces and having modules communicate with each other using those interfaces.

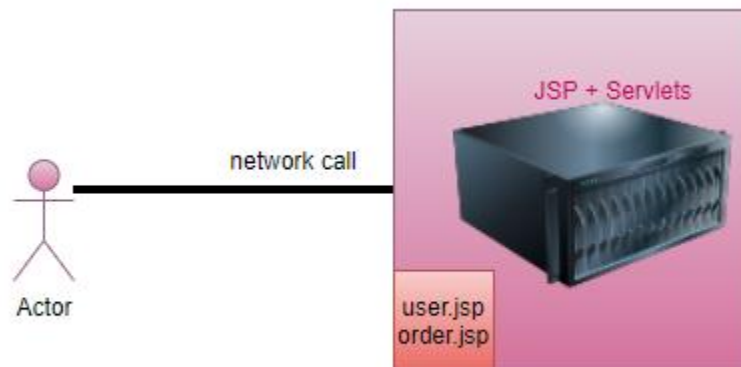


This becomes a good foundation for the complete decomposition and service extraction form monolith.

Decouple the UI and the Backend Business Logic



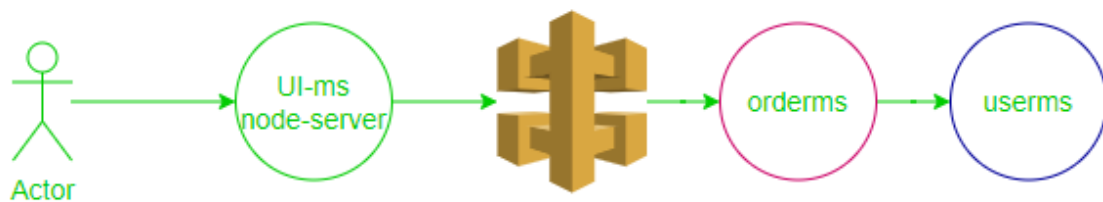
In old style when user clicks from one page to another there was a server call



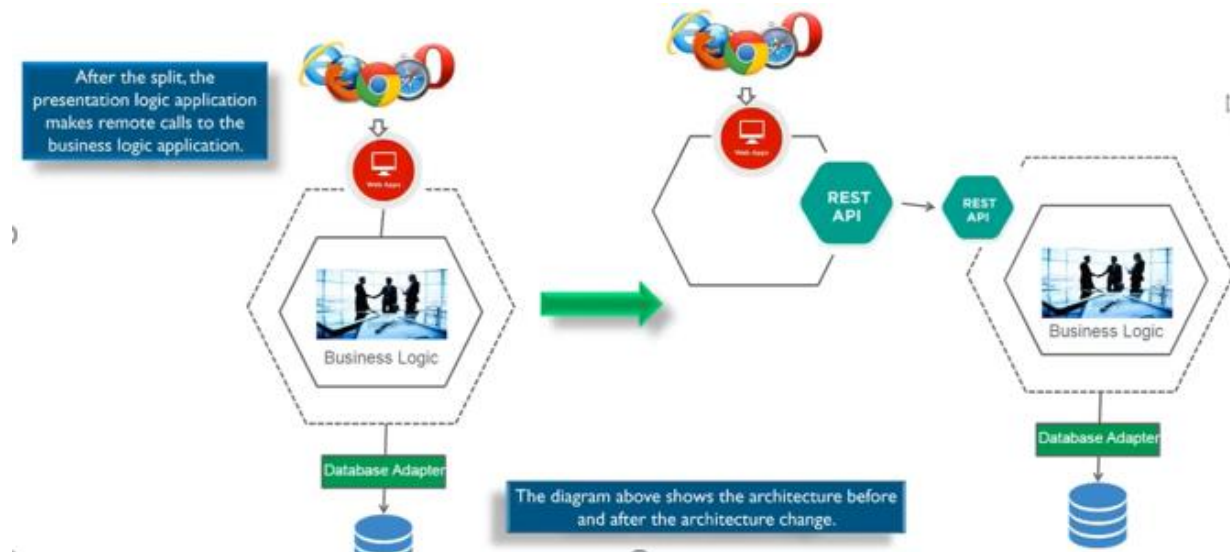
Single Page Application-> Angular, React, ViewJS

When user clicks from one page to another page, don't need of server call, pages are already available at client side, they need server call only for data.

e.g., present as ms



Business logic and Presentation



Mud ball to Services

Extract the services out of the legacy code: After decomposing the mud ball under various categories of deployment (new functionality, Presentation, business logic), we need to extract the services out of the legacy code.



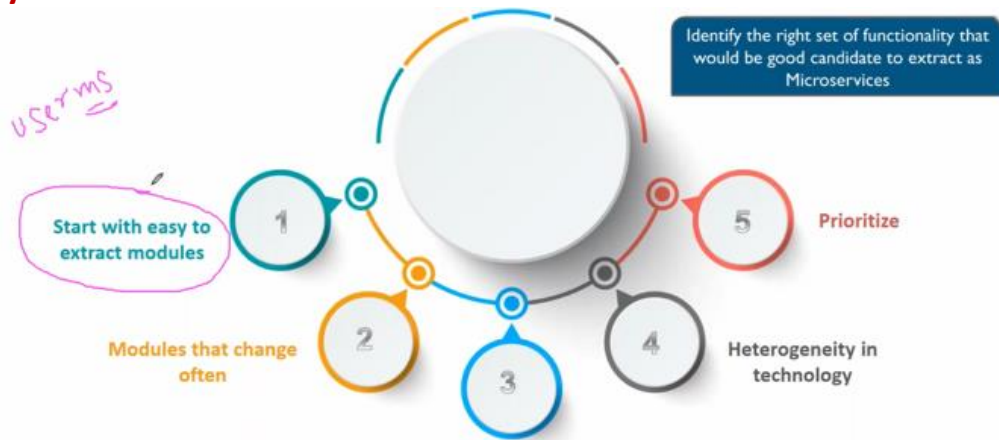
Every time we extract a functionality as a service, the **mud ball will start shrinking** and a point will come when Monolith will die or will no more be a monolith. This is the final state of the transition to a microservices architecture.

Use **DDD** to define bounded context and break the complex system into smaller contexts and build the relationships between them where possible.

DDD

steps:

Identify candidates

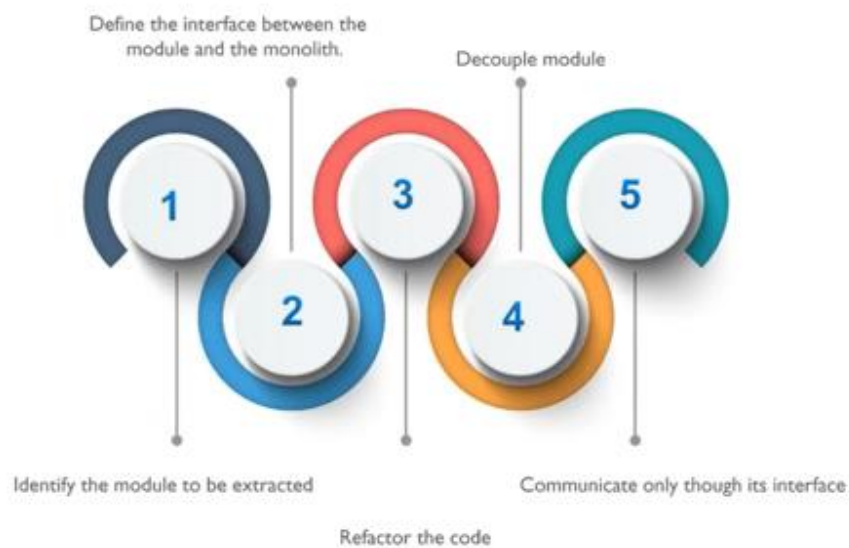


Service Extraction

It is a tedious process due to the dependencies and relationships among domain objects is tightly coupled.

Once you understand the process, services extraction complexity keep reducing.

Service Extraction- step1



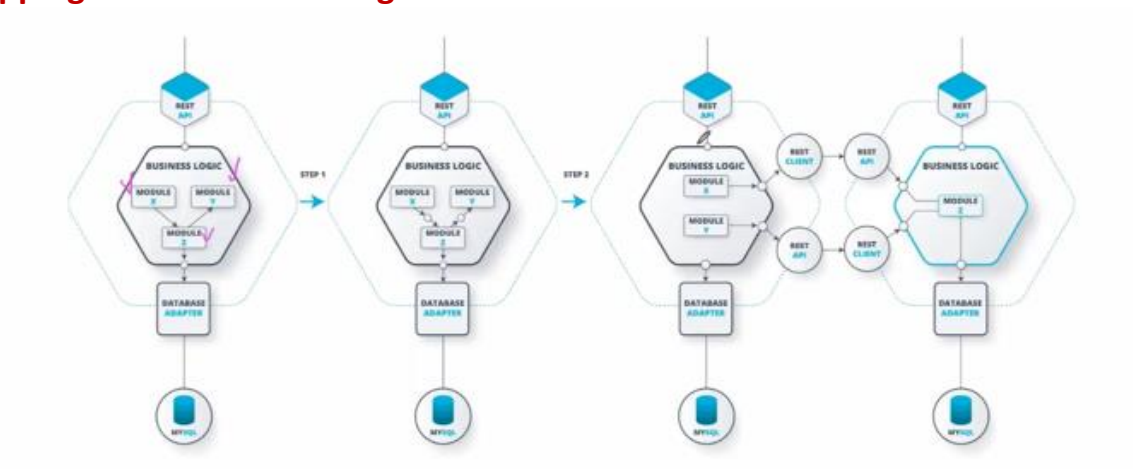
Service Extraction- step2

Turn the Module into a Service: One module is isolated with interface, we need to turn it in a service, that would include following:

- **Implement a rest interface** so the module can be accessed remotely by monolith
- **Build the glue code** for the monolith to talk to service and vice versa.
- Use the **data access techniques** described earlier to meet the data flow needs between service and monolith.

In next slide we will show how the module Z is extracted out using these two steps and the architecture transitions happens

Popping small balls from big mud



Monolith Code

```
public class ClassX {  
    ClassZ classZ; // Composition Has-A relationship  
  
    public void process() {  
        classZ.callMethod();  
    }  
}  
  
public class ClassZ {  
    void callMethod() {  
        // some processing  
    }  
}
```

Refactoring

Step1: first convert HAS-A relationship in to interface (IS-A)

```

public class ClassX {
    ClassZ classZ; // Composition Has-A relationship

    public void process() {
        classZ.callMethod();
    }
}

public interface ClassZ {
    void callMethod();
}

public class ClassZLocal implements ClassZ {

    void callMethod() {
        // some processing
    }
}

```

Step2: For refactor create one more class ClassZRemote

```

public interface ClassZ {
    void callMethod();
}

public class ClassZRemote implements ClassZ {

    void callMethod() {
        // Remote processing
        restTemplate.getForObject("z-ms", Object.class);
    }
}

```

Step3: By default, ClassZLocal defined as a @Primary

```

@Primary
public class ClassZLocal implements ClassZ {

    void callMethod() {
        // some processing
    }
}

```

Step4: Once ms is created, update the annotation @Primary to ClassZRemote.

```

@Primary
public class ClassZRemote implements ClassZ {

    void callMethod() {
        // Remote processing
        restTemplate.getForObject("z-ms", Object.class);
    }
}

```

Key Architecture Decisions



When: Benefits we get from ms

Agility

Light Weight

HA

Heterogeneity

Cherry-Pick Scalability

Maintenance

Highly Scalable

How:

DDD to Identify the Bounded Context

<3-5 ms calls

Benchmarking

Ready:

ADAPT

Cost (High)-> License + Developer

Many teams (2 Pizza team)

DevOps Culture

When to use Microservices?

